

Lecture 5: Tokenization

Week 2: From Characters to Meaningful Units

LLM Course

By the end of this lecture, you will:

1. Understand what tokenization is and why it matters
2. Explain the limitations of word-level tokenization
3. Describe how subword tokenization works (BPE, WordPiece, SentencePiece)
4. Compare different tokenization methods and their trade-offs
5. Implement and experiment with different tokenizers using HuggingFace

Central question: How do we break text into meaningful units for AI models? [?]

What is Tokenization?

Definition: Converting text into smaller units (tokens)

Why tokenize?

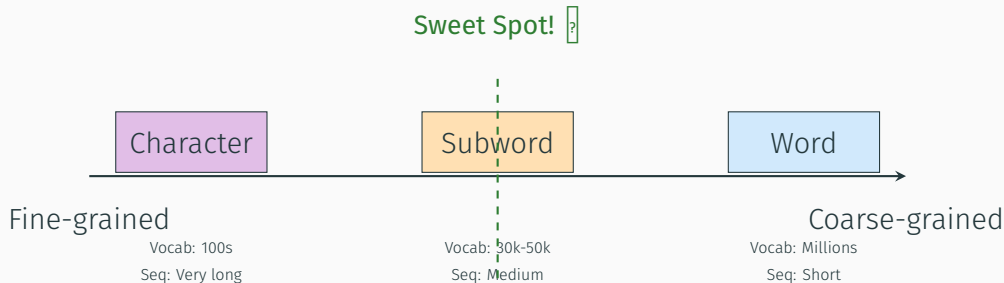
- Neural networks process numbers, not text
- Need discrete units to create vocabulary
- First step in any NLP pipeline

What can tokens be?

- **Characters:** a, b, c, ... (very fine-grained)
- **Words:** "hello", "world" (intuitive but limited)
- **Subwords:** "un", "happiness" (sweet spot! )
- **Bytes:** 0-255 (most fine-grained)

"The granularity of tokenization determines what a model can learn"

The Tokenization Spectrum [?]



Key trade-off: Vocabulary size vs. sequence length

Why Not Just Split on Spaces? [?]

The naive approach:

`text.split(" ")` → Done! [?]

Problem 1: Vocabulary explosion [?]

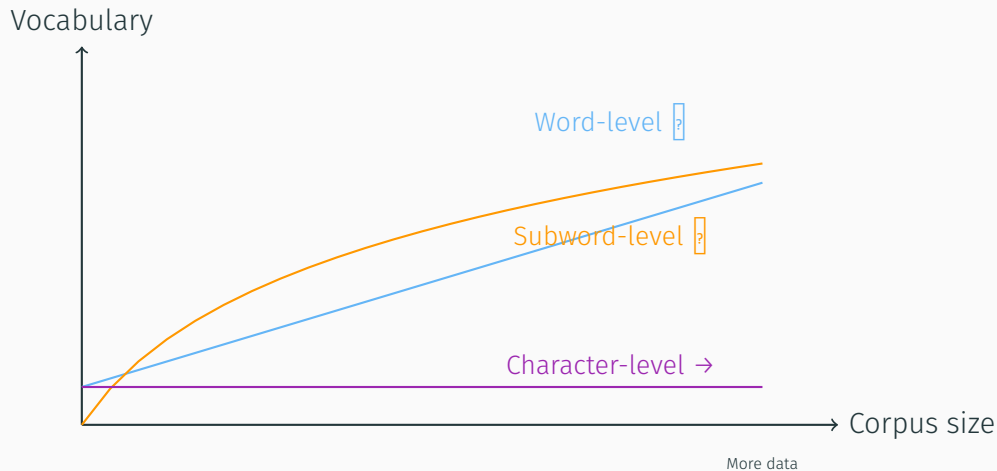
- English: 170,000 words in active use
- Each inflection counts separately: "run", "runs", "running", "ran"
- Compounds: "ice cream", "icecream", "ice-cream"

Problem 2: Unknown words [?]

- Out-of-vocabulary (OOV): "supercalifragilisticexpialidocious"
- New words: "COVID-19", "selfie", "blockchain"
- Typos and misspellings: "teh" instead of "the"

Problem 3: No spaces in some languages! [?]

Word-level Limitations Illustrated [?]



Observation: Word-level vocabulary keeps growing!

The OOV Problem in Action ?

```
# Simple word-level vocabulary
vocab = {"hello", "world", "the", "cat", "sat"}

def tokenize_word_level(text, vocab):
    tokens = text.lower().split()
    result = []
    for token in tokens:
        if token in vocab:
            result.append(token)
        else:
            result.append("<UNK>") # Unknown token
    return result

# Example
text = "Hello! The cat jumped"
tokens = tokenize_word_level(text, vocab)
```

The Solution: Subword Tokenization [?]

Key insight: Most words are made of smaller meaningful pieces!

Examples:

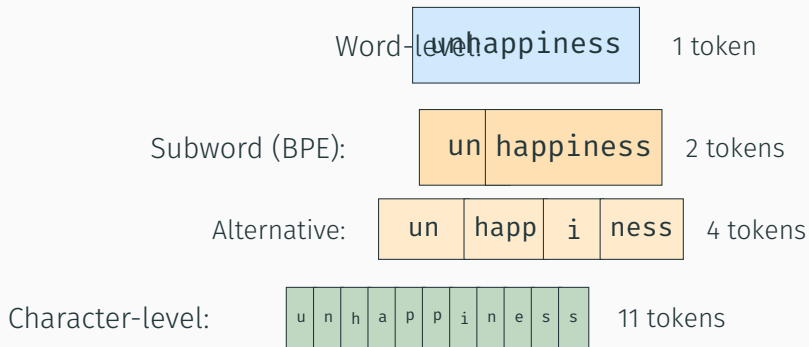
- "unhappiness" = "un" + "happiness"
- "preprocessing" = "pre" + "process" + "ing"
- "antiestablishmentarianism" = "anti" + "establish" + "ment" + "arian" + "ism"

Benefits:

- [?] Fixed, manageable vocabulary (30k-50k tokens)
- [?] Handle unknown words by breaking into known parts
- [?] Capture morphology: prefixes, suffixes, stems
- [?] Language-agnostic: works for any language!

How? Learn common character sequences from data! [?]

Visual: How "unhappiness" Gets Tokenized ?



Trade-off: More tokens = longer sequences vs. smaller vocabulary

Byte-Pair Encoding (BPE) [?]

Original use: Data compression (1994)

Adapted for NLP: Sennrich et al. (2016)

Core idea: Iteratively merge the most frequent pair of characters/subwords

Algorithm:

1. Start with character-level vocabulary
2. Count all adjacent pairs in corpus
3. Merge most frequent pair → create new token
4. Repeat until desired vocabulary size

Used by: GPT, GPT-2, GPT-3, RoBERTa, BART, many others!

[?] **Reference:** Sennrich, Haddow, & Birch (2016). Neural Machine Translation of Rare Words with Subword Units. *ACL*

BPE Example: Step by Step

Training data: "low low low lower lower newest newest newest newest widest"

Initial tokens (characters):

l, o, w, e, r, n, s, t, i, d

Iteration 1: Most frequent pair = e, s (appears 4 times)

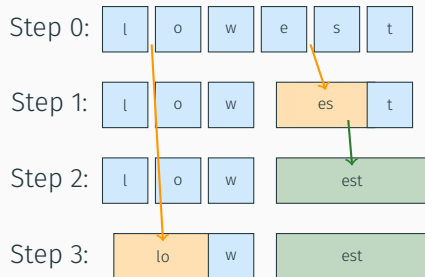
- Merge → new token: es
- Vocabulary: l, o, w, e, r, n, s, t, i, d, es

Iteration 2: Most frequent = es, t

- Merge → new token: est
- Vocabulary: l, o, w, e, r, n, s, t, i, d, es, est

Iteration 3: Most frequent = l, o

BPE Visualization ?



Result: "lowest" → [low, w, est]

Captures common patterns without explicit linguistic rules!

```
from transformers import AutoTokenizer

# GPT-2 uses BPE
tokenizer = AutoTokenizer.from_pretrained("gpt2")

text = "I'm learning about tokenization!"

# Tokenize
tokens = tokenizer.tokenize(text)
print("Tokens:", tokens)
# Output: ['I', "'m", 'Ġlearning', 'Ġabout', 'Ġtoken', 'ization', '!']
# Note: 'Ġ' represents space

# Get token IDs
token_ids = tokenizer.encode(text)
```

WordPiece Tokenization

Similar to BPE, but with a twist!

Key difference: Instead of merging most *frequent* pair, merge pair that maximizes *likelihood* of training data

Merge criterion:

$$\text{score}(x, y) = \frac{\text{freq}(xy)}{\text{freq}(x) \times \text{freq}(y)}$$

Intuition: Prefer merging pairs that are "surprisingly common" together

Used by: BERT, DistilBERT, Electra

Special tokens:

- ## prefix for continuation subwords
- Example: "playing" → ["play", "##ing"]

WordPiece Example with BERT ?

```
from transformers import AutoTokenizer

# BERT uses WordPiece
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

text = "I'm learning about tokenization!"

tokens = tokenizer.tokenize(text)
print("Tokens:", tokens)
# Output: ['i', "'", 'm', 'learning', 'about', 'token', '##ization', '!', '']
#
#                                     ^^^ Note
#   the ##

# Compare with longer word
text2 = "unhappiness"
```

SentencePiece: Language-Agnostic Tokenization

Problem with BPE/WordPiece:

- Assume text is pre-tokenized (words separated by spaces)
- Doesn't work for languages without spaces (Chinese, Japanese, Thai...)

SentencePiece solution:

- Treats input as raw character stream (no pre-tokenization!)
- Spaces are just another character: `▯`
- Truly language-agnostic

Two algorithms:

1. **Unigram Language Model:** Start with large vocabulary, iteratively remove tokens
2. **BPE:** Same as before, but on raw text

SentencePiece in Action ?

```
from transformers import AutoTokenizer

# T5 uses SentencePiece
tokenizer = AutoTokenizer.from_pretrained("t5-small")

text = "I'm learning about tokenization!"

tokens = tokenizer.tokenize(text)
print("Tokens:", tokens)
# Output: ['I', "'", 'm', 'learning', 'about', 'token', 'ization', '!']
#           ^^^ Note the _ for spaces

# Works seamlessly with other languages!
text_chinese = "?????"
tokens_cn = tokenizer.tokenize(text_chinese)
```

Tokenization Methods Comparison

Method	Approach	Used By	Key Feature
BPE	Frequency-based merging	GPT-2, GPT-3, RoBERTa	Simple, effective, bottom-up
WordPiece	Likelihood-based merging	BERT, DistilBERT	Statistically principled
SentencePiece (Unigram)	No pre-tokenization, top-down pruning	T5, ALBERT, mT5	Language-agnostic, multilingual
SentencePiece (BPE)	No pre-tokenization, bottom-up	XLNet, mBART	Combines BPE with raw text

HuggingFace Resources:

- Chapter 2.4: Behind the Pipeline - Tokenizers
- Chapter 6: Tokenizers (full chapter)

Comparing Tokenizers Side-by-Side ?

```
from transformers import AutoTokenizer

text = "The unhappiest researchers couldn't preprocess data!"

models = {
    "GPT-2 (BPE)": "gpt2",
    "BERT (WordPiece)": "bert-base-uncased",
    "T5 (SentencePiece)": "t5-small"
}

for name, model_name in models.items():
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    tokens = tokenizer.tokenize(text)
    print(f"\n{name}:")
    print(f"    Tokens ({len(tokens)}): {tokens}")
```

Special Tokens ?

Most tokenizers add special tokens for specific purposes:

Token	Purpose	Model
[CLS]	Classification token	BERT
[SEP]	Separator between segments	BERT
[PAD]	Padding to same length	Most models
[UNK]	Unknown/rare tokens	Most models
[MASK]	Masked token for training	BERT
<s>, </s>	Start/end of sequence	GPT-2, T5
< endoftext >	Document boundary	GPT-2

Example usage:

[CLS] The cat sat [SEP] The dog ran [SEP]

Working with Special Tokens ?

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

# Special tokens are automatically added
text = "Hello world"
encoded = tokenizer(text, return_tensors="pt")
print("Input IDs:", encoded['input_ids'])
# [101, 7592, 2088, 102]
#   ^^^           ^^^ [CLS] and [SEP] added!

# Decode with/without special tokens
full_decode = tokenizer.decode(encoded['input_ids'][0])
print("With special:", full_decode)
# Output: "[CLS] hello world [SEP]"
```

Vocabulary Size Trade-offs ?

Smaller vocabulary (e.g., 10k tokens):

- ? Faster training (smaller embedding matrix)
- ? Less memory
- ? Longer sequences (more subwords per word)
- ? May lose semantic information

Larger vocabulary (e.g., 100k tokens):

- ? Shorter sequences (closer to word-level)
- ? Better semantic preservation
- ? Slower training (larger embeddings)
- ? More memory required

Sweet spot: 30k-50k tokens for most models

Common issues to watch out for:

1. Tokenizer-model mismatch

- Always use the tokenizer that matches your model!
- GPT-2 tokenizer $\neq$ BERT tokenizer

2. Maximum sequence length

- BERT: 512 tokens | GPT-2: 1024 | GPT-3: 2048
- Text gets truncated if too long!

3. Case sensitivity

- `bert-base-uncased` lowercases everything
- `bert-base-cased` preserves case

4. Rare words $\rightarrow$ many tokens

- "antidisestablishmentarianism" $\rightarrow$ 10+ tokens
- Can hit sequence limit faster than expected!

5. Special characters

- Emoji, Unicode, accents may be split unexpectedly

Debugging Tokenization ?

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("gpt2")

text = "The antidisestablishmentarianism debate continues! ?"

# Tokenize and see details
tokens = tokenizer.tokenize(text)
print(f"Tokens ({len(tokens)}): {tokens}\n")

# Get IDs and decode
token_ids = tokenizer.encode(text)
for i, (token, token_id) in enumerate(zip(tokens, token_ids)):
    decoded = tokenizer.decode([token_id])
    print(f"{i:2d}. ID {token_id:5d} | Token: {token:20s} |  
        Decoded: {decoded}")
```


Reminder from infant learning (Saffran et al., 1996):

- Babies track statistical regularities in speech
- Identify word boundaries from transitional probabilities
- No explicit rules—just patterns from exposure!

Parallel with subword tokenization:

- BPE: Merge frequent character pairs → discover common morphemes
- Infants: Track frequent syllable pairs → discover words
- Both are *data-driven*, not rule-based!

Key difference:

- Infants: Online, real-time, multimodal (sound + context)
- BPE: Batch, text-only, no grounding

Infant Learning:

Input stream:

`bidakupadotigolabu...`

Learn high-probability sequences:

- `bi-da-ku` (word)
- `pa-do-ti` (word)

Detect low-probability boundaries:

- `ku | pa` (boundary)

BPE Learning:

Input corpus:

`low low lower...`

Merge high-frequency pairs:

- `l+o → lo`
- `lo+w → low`

Build vocabulary:

- `low, lower, lowest`

Both use distributional statistics to discover structure! ?

Experiment with different tokenizers!

1. Choose 3 models: GPT-2, BERT, T5
2. Test on diverse texts:
 - Standard English: "The cat sat on the mat"
 - Complex words: "antidisestablishmentarianism"
 - Contractions: "I'm, you're, won't"
 - Typos: "teh qiuck brown fox"
 - Emoji: "I love this!  
 - Other languages: "
3. Compare results:
 - Number of tokens
 - How words are split
 - Handling of unknown/rare words
4. Reflect:
 - Which tokenizer works best for your use case?

Discussion Questions

1. Linguistics vs. Statistics:

- BPE discovers morphemes (un-, -ing, -ness) without linguistic rules. Is this "learning" morphology, or just pattern matching?

2. Cross-lingual tokenization:

- Should we use the same tokenizer for all languages? What are the trade-offs?

3. Semantic preservation:

- Does breaking "unhappy" into ["un", "happy"] preserve meaning? What about "butterfly"?

4. Human vs. machine:

- Humans don't consciously tokenize words. Why do machines need to?

5. Future directions:

- Will we move toward character-level or byte-level models that don't need tokenization?

Foundational papers:

- Sennrich, Haddow, & Birch (2016). Neural Machine Translation of Rare Words with Subword Units. *ACL*.
 - Introduced BPE for NLP
- Kudo & Richardson (2018). SentencePiece: A simple and language independent approach. *EMNLP*.
 - Language-agnostic tokenization
- Wu et al. (2016). Google's Neural Machine Translation System. *arXiv*.
 - WordPiece algorithm

HuggingFace resources:

- Chapter 2.4: Tokenizers
- Chapter 6: Tokenizers (detailed)

Key Takeaways

1. **Tokenization is fundamental**
 - Bridges raw text and neural networks
2. **Word-level has major limitations**
 - OOV problem, vocabulary explosion, language-dependent
3. **Subword tokenization is the sweet spot**
 - Balanced vocabulary size and sequence length
4. **Different methods, similar principles**
 - BPE: frequency-based | WordPiece: likelihood | SentencePiece: language-agnostic
5. **Statistical learning connects humans and machines**
 - Both discover structure from distributional patterns
6. **Always match tokenizer to model!**
 - Critical for correct predictions

Next lecture: POS Tagging & Sentiment Analysis

How tokenization connects:

- POS tagging: Token-level classification
- Sentiment: Sequence-level classification
- Both depend on good tokenization!

Prepare by:

- Experimenting with HuggingFace tokenizers
- Thinking about: How does token granularity affect downstream tasks?
- Exploring: Tokenizer artifacts and their impact

Questions? 

Next: Lecture 6 - POS Tagging & Sentiment Analysis